

Model context protocol for MBSE: a unified interoperable agentic framework for systems modelling

Mahmoud Bazzal^{1,2,✉}, Simon Schleifer^{3,2}, Adriana Lungu², Benjamin Kruse², Stefan Goetz³, Christian Koldewey^{1,4}, Denis Tissen¹, Sandro J. Wartzack³ and Roman Dumitrescu^{1,4}

¹ Heinz Nixdorf Institute, Paderborn University, Germany, ² Audi AG, Germany,

³ Engineering Design (KTmfk), Friedrich-Alexander Universität Erlangen-Nürnberg, Germany,

⁴ Fraunhofer Institute for Mechatronic Systems Design, Paderborn, Germany

✉ mahmoud.bazzal@hni.uni-paderborn.de

ABSTRACT: With the introduction of SysML v2—featuring a new metamodel and standardized APIs—new opportunities arise for automating MBSE tasks through generative AI. However, existing LLM approaches lack interoperability due to limited support for complex SysML v2 API operations. This paper presents a framework for implementing Model Context Protocol (MCP) servers for SysML v2, enabling LLMs to interact with system models via the API. This framework provides a foundation for agentic MBSE by allowing automation of modeling tasks while maintaining interoperability across tools and generative AI systems.

KEYWORDS: model-based engineering (MBE), artificial intelligence (AI), model context protocol, agentic AI

1. Introduction

Product development, particularly in the automotive industry, faces increasing complexity due to customer demands for new functionalities and intense competition, driving pressure to reduce time-to-market. To manage this complexity and enhance product quality, industry increasingly adopts Model-Based Systems Engineering (MBSE) (Walden, 2015), beginning with functional vehicle architecture development. Effective MBSE integrates methodology, tool, and language: the methodology defines semantic model development, the tool enables model creation, and the language specifies syntactic structure (Delligatti, 2014). The Systems Modeling Language (SysML) is widely used; its first version supports requirements, structure, and behavior modeling via diagrams. Recently published SysML v2 addresses limitations such as reliance on the Unified Modeling Language (UML) metamodel and limited interoperability (Bajaj et al., 2022). It introduces a new metamodel, standardized APIs, and a textual representation alongside the familiar graphical views (Bajaj et al., 2022).

Manual modelling is often seen as time-consuming but could be accelerated with AI, enabling architects to focus on non-routine tasks. Semantic text processing is required to convert natural language requirements into diagrams and refine models. Large Language Models (LLMs), together with SysML v2's textual representation and standardized APIs, open new possibilities for AI-assisted MBSE. Yet, custom LLM implementations limit interoperability. This work proposes a universal framework that integrates LLMs with SysML, allowing agentic AI to perform SysML v2 modelling tasks via natural language prompts, reducing the need for manual model creation and editing.

System architects frequently perform repetitive SysML modeling tasks throughout a product's lifecycle. Generative AI can handle these tasks in an agentic manner, allowing architects focus on system design rather than modeling details, thereby increasing productivity and reducing time-to-market.

Model Context Protocol (MCP) is a new approach to enable the automation of many tasks requiring both cognitive and semantic evaluation by allowing LLMs to use software tools. Even though the use of MCP has been emerging in many industries, it has yet to be used for enabling modeling tasks in MBSE. This gap has existed so far due to multiple reasons such as the complexity of the meta model of the language being used, the proprietary nature of modeling tools, and focus on graphical modeling in many of the tools being used in the industry. These challenges have been partially addressed with the introduction of SysML v2, however, the complexity of the new metamodel leads to difficulties in implementing MCP tools that interact with it. Therefore, developing such tools has not been studied yet. Based on this need, a research question emerges: **How to enable agentic approaches to automate MBSE modeling tasks, while maintaining the interoperability of agents with different modeling tools and generative AI components?**

To answer this question, we split it into two aspects, namely agentic task automation in MBSE, and interoperability of agents performing these tasks. Therefore, we address each of these aspects with a requirement that must be satisfied by the artifact developed in this work. These requirements are formulated as follows:

An agentic approach supporting MBSE should be defined. Namely, the modeling tasks that should be supported by an MBSE agent. Additionally, interoperability between the developed tools that an agent would be able to use, and a variety of LLMs and other generative AI components that could be used with it, should be guaranteed.

In the remainder of this paper, we present a framework that satisfies these requirements, thereby providing an answer to the research question presented in this section.

The remaining sections are organized as follows: [Section 2](#) reviews the state of the art in the fields of natural language processing, agentic AI approaches to task automation, and modelling with SysML v2. [Section 3](#) describes the research methodology we use in the publication. [Section 4](#) introduces the proposed methodological framework for agentic MBSE in detail, which is then illustrated through two exemplary case studies in [Section 5](#). [Section 6](#) provides a critical discussion of the findings, and [Section 7](#) concludes the paper with a summary of the work.

2. State of the art

To formally identify the research objective that satisfies the requirements formulated in [section 1](#), we examine the existing state of the art related to each of these requirements. This examination of the state of the art is then used to clarify the research gap addressed by this work.

The first requirement can be addressed by defining the capabilities the developed tooling framework should possess. Such a framework should be capable of a) creating, b) manipulating, and c) querying a given system architecture model. Most modelling tasks can be distilled into a combination of these capabilities. These tasks can then be combined to model a system as needed. Thus, enabling the system architect to focus on higher level tasks.

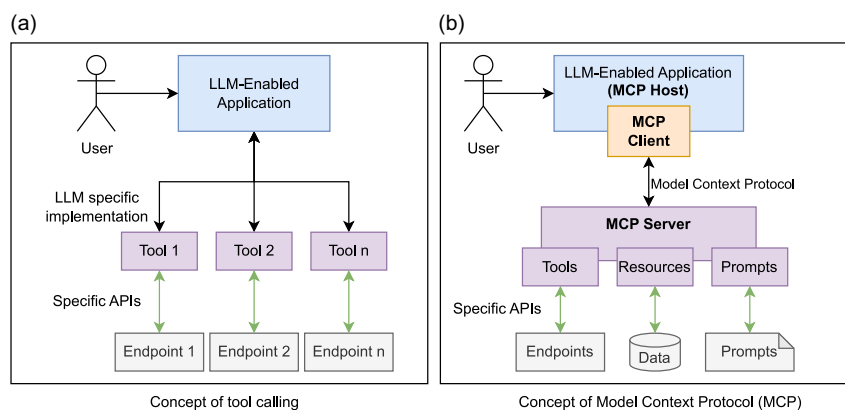


Figure 1. Overview of conceptual architecture of tool calling and model context protocol (MCP)

LLMs significantly expand semantic text processing and generation beyond Natural Language Processing (NLP). LLMs are task-agnostic language models, they perform downstream NLP tasks (e.g. classification, summarization and generation) without task-specific training due to their architectural,

based on the decoder part of the attention mechanisms (Vaswani et al., 2017), enabling autoregressive text generation (Jurafsky & Martin, 2024). The output of LLMs depends on the provided input, i.e., the prompt and previously generated tokens, making prompt engineering essential. Adding examples— i.e., in-context learning (one-shot or few-shot) (Brown et al., 2020)—can significantly improve accuracy. However, outputs remain constrained by model parameters and limited context windows, restricting integration of large domain-specific data. To avoid costly finetuning of the LLM, solutions such as Retrieval Augmented Generation (RAG) can be utilized to infer new data sources (Lewis et al., 2020). In the context of MBSE, this can be documentation on modelling methods, or document-centered artefacts occurring during product development. Another option is tool calling, which enables the LLM to invoke provided software functions both to gather specific information and to directly interact with software solutions, typically via specific APIs (Hou et al., 2025). However, this concept requires individual implementations for different LLMs and is thus limited in its applicability, see Figure 1a.

In addition to the generic generative and agentic AI approaches available in the state of the art, works addressing the use of GenAI in MBSE exist. (Mollahassani et al., 2025) represented a framework for reusing partial MBSE models generated with GenAI, without addressing the possibility of developing agents that can overtake given modeling tasks in general. Additionally, (Poulsen et al., 2025) discusses future potential potentials of performing MBSE tasks with AI support without addressing any specific framework that could be instantiated to automate modeling tasks, regardless of the agentic or pure GenAI nature of such a framework.

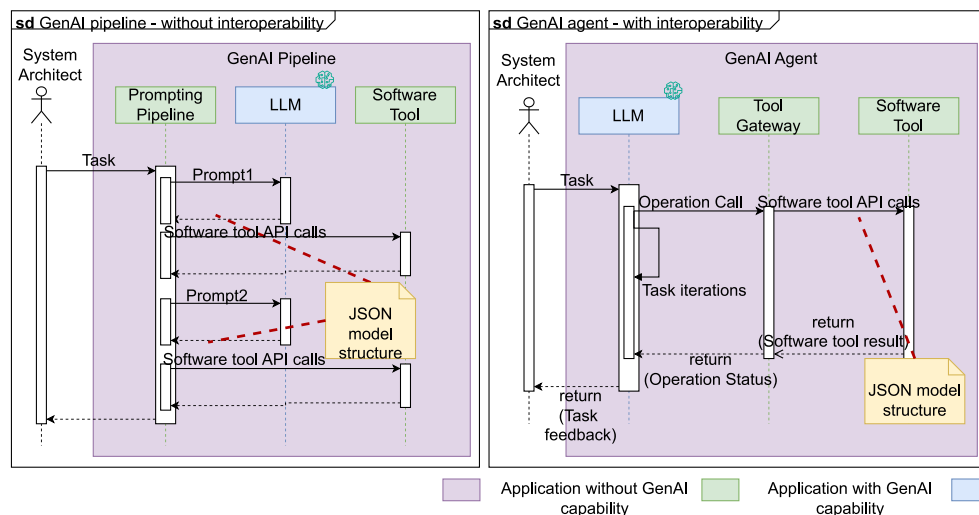


Figure 2. Overview of the use of agentic modelling approaches without a tool gateway (left) and with a tool gateway and tool calling (right)

The second requirement can be addressed by considering a) the inherent interoperability of SysML v2, and its standardized API, and b) the general interoperability of LLMs.

With regards to SysML v2 and its API, given that modeling tools implement that API, they can not only seamlessly communicate with each other, but also with a central database containing the system model, which can be used throughout the lifecycle of the system.

When considering interoperability across LLMs, directly connecting software components to generative AI applications requires LLM-specific implementations for each endpoint. Switching models demands reimplementing, creating fragmented architectures, increasing maintenance overhead, and introducing inconsistencies in agent behavior—similar to human collaborators using incompatible methods. Tight coupling also assumes a fixed LLM and agentic strategy, limiting experimentation with prompting or alternative agent configurations. These limitations motivate an intermediary interface layer between LLMs and tools, enabling a more robust agentic approach (Figure 2).

In a non-interoperable architecture, single prompts are issued directly to an LLM, and processed synchronously (Figure 2 left-hand side), requiring the LLM to generate explicit API calls at every step. This is unsafe, as malformed or schema-invalid outputs—often due to unreliable JSON or structured data generation—can break downstream systems. A tool gateway between the LLM and the underlying software (Figure 2 right-hand side) abstracts task-specific operations away from the LLM, enabling a

clear separation of concerns: the LLM handles reasoning, while the gateway reliably performs all structured operations. In a non-interoperable approach, a single malformed output can corrupt an entire multi-step process and force a full restart. In an interoperable, agentic architecture, the LLM can instead recover—re-planning, retrying, or invoking alternatives—since it no longer must generate low-level, schema-correct API calls.

Within SysML v2 modeling, this generalized agentic-interoperability approach directly addresses the challenges of having LLMs produce SysML v2 API requests. The SysML v2 API requires precise, schema-valid JSON structures to create, modify, or query elements within a model. When an LLM must generate these structures directly, errors are common and can invalidate or corrupt the model. By abstracting SysML v2 operations behind a tool gateway, the LLM interacts only with high-level modeling intents, while the gateway translates these intents into reliable SysML v2 API calls. This preserves model integrity, enables consistent tool behavior across different LLMs, and allows experimentation with various prompting strategies or agent configurations without risking invalid SysML v2 requests or forcing complete reconstruction of the model.

Other efforts have been made to enhance LLMs' ability to generate valid structured output that can be used to describe models. (Bader et al., 2024) attempted to do this by finetuning LLMs to generate XML structures. (Bader et al., 2024) aims to provide a conversation-driven generative AI MBSE experience. While it addresses the interaction between the user and an LLM-driven modeling application, it does not address the possibilities of agentic completion of modeling tasks, and it only partially addresses interoperability insofar that the LLM generated output is a machine-readable format, namely XML.

An implementation of the Model Context Protocol (MCP) can serve as a tool gateway (Figure 2 left-hand side) and consists of three parts (Figure 1b). The MCP host—an AI application such as GitHub CoPilot in VS Code—offers various LLMs and coordinates MCP clients, which gather context from one-to-one-connected MCP servers. Clients retrieve all server functions, select which to call, and return results to the host. Servers provide three functionalities: tools for operations invoked by the LLM (e.g., adding relationships to a system model), resources for data retrieval (e.g., retrieval of all model elements), and user-invoked prompts, that serve as templates or best-practice guidance, e.g., checking model conflicts via LLM assistance (Hou et al., 2025). In addition to MCP, other frameworks enabling agentic interoperability of deterministic software with LLMs are surveyed and compared on the implementation level in (Ehtesham et al., 2025). Addressing those implementation details is out of scope of this paper, since up to this point, the integration of MCP in MBSE is generally unstudied.

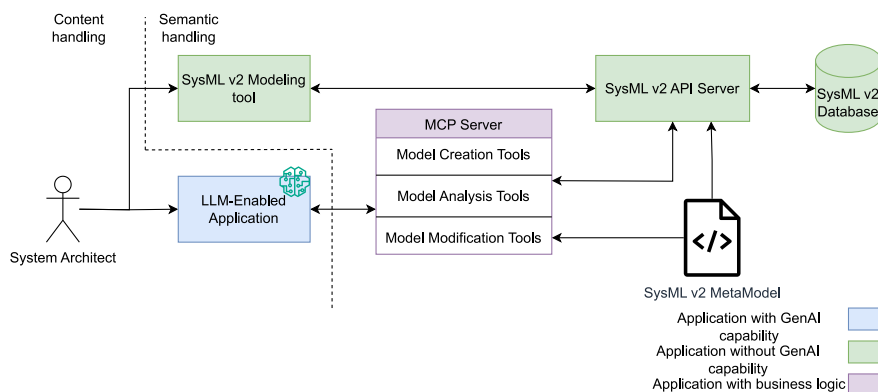
In this section, we explored the state of the art in generative AI, agentic approaches, and SysML v2. However, to the best of the authors' knowledge, there have been no published works so far that address either one of the two requirements presented in section 1 to a full extent, or their combination. Therefore confirming the novelty of the contribution presented in this work.

3. Research methodology

Given the research question proposed in section 1 and the state of the art we explore in section 2, we structured the research leading to this publication according to the Design Science Research Methodology (DSRM) according to (Peppers et al., 2007). The first phase of the DSRM recommends identifying the problem which we do through proposing the research question and defining the requirements of the artifact addressing this question in section 1. The second phase includes identifying the design objective. This is done through performing an analysis of existing state of the art and showing the existing research gap between generic research regarding agentic and interoperable AI, and the current usage of GenAI in MBSE (section 2). Subsequently, the research objective is then identified as **the design of a software artifact that enables the implementation of agents to automate the modelling of system architecture using SysML v2**. The research artifact addressing this goal is then implemented as described in section 4. The demonstration step in DSRM is then conducted through two case studies showing the application of this agentic framework for a) use case creation, and b) signal optimization in a given system architecture (section 5.) We then perform an initial evaluation of this work by discussing the case studies used for demonstrating the framework (section 6,) including the benefits, limitations, and possibilities of future work correlating to the contribution of this paper. The last phase is communication of results which is done in this paper.

4. SysML v2 MCP framework

In this section, we describe the implementation of an MCP server with different classes of tools as shown in Figure 3. Using this methodology, the system architect can interact with an LLM enabled application such as a chat bot, or a user interface that allows the execution of a modeling task. Note that details of such an application are out of the scope of this paper, such as the ability to determine the modeling context being given to the LLM, the prompting technique used, and the architecture of such an LLM based application. LLMs that support MCP can then decide dynamically to call tools provided by the MCP server as needed, while performing the task. These tools are provided during tool discovery when executing the inference on a provided prompt. A tool can execute as a self-contained operation that implements an input-output relationship. When calling a tool, the LLM inquiries about the needed arguments for an operation, and generates those arguments either from the provided context, or by asking the user. The tool implementing the SysML v2 logic uses the appropriate metamodel provided and the SysML v2 API, to manipulate the database containing the model, and finally providing feedback on the requested operation back to the LLM.



The overall usage of MCP as a tool gateway accessed by an LLM

Taking a closer look at the MCP server, we show the framework used to implement it in Figure 4. The server provides a tool gateway that exposes all developed tools to the MCP interface. Each one of these tools inherits the class *AbstractSysMLV2Tool*, which handles the concrete SysML v2 type provided. Additionally, each developed tool must differentiate between mandatory and optional parameters for achieving the operation that is implemented. The tool can then use one or more concrete factories to create or manipulate model elements. The implementation of these model elements is derived dynamically from the JSON schemas describing the SysML v2 metamodel. Each of these concrete factories need to have the ability to create, modify, and serialize objects for the concrete metamodel element being exposed by an MCP tool. This follows the factory pattern to programmatically handle SysML v2 model elements. Finally, the tool logic can perform operations on the database containing the model by calling a combination of operations provided by the SysML v2 API client. To enable non-modelling operations, such as creating branches, and handling multiple projects, we also expose these operations to the MCP interface as tools. This can enable the LLM-based application to be used for branch-based development.

This framework benefits the system architect by separating the concerns of semantic and content processing. Processing the model content is handled by the system architect with the LLM and processing the model semantic is automated in the MCP server application. It also benefits the developer of tools enabling the LLM, by using the factory pattern, for example, to abstract the handling of SysML v2 model elements from the “business logic” being implemented by each tool.

In addition to the separation of concerns, a question arises in terms of the granularity of the tools being implemented. Implementing tools that handle multiple modeling steps at once, or an entire modeling task, would defeat the purpose of an MCP server, since such a tool represents the entire task which an agent is designed to perform, which would reduce the flexibility of the user and the LLM to guide the task and perform it iteratively. However, creating only elementary tools designed to perform atomic functions leads to a higher number of calls and therefore lower scalability, in terms of both time and the amount of

tokens – and by extension the computational resources and cost – required to perform a given modeling task.

To address this granularity problem, we identify three classes of tools that should be implemented to enable the automation of modelling tasks. Namely, i) model creation tools, ii) model modification tools, and iii) model analysis tools. In the case of model creation tools, we differentiate between atomic and non-atomic elements that should have dedicated tools. For example, a metamodel element such as a package should have a dedicated tool to create it, while a documentation attached to the package is created as a part of package, but does not have its own tool that the LLM can call. When creating a new model element, the tool expects the LLM to fill in all the required attributes for that model element. These attributes are calculated automatically by referencing the SysML V2 metamodel as shown in Figure 4. Similarly, model modification tools create changes in a given model element holistically rather than by accessing specific attributes in that element. Since model analysis tools do not require any creation or modification of model elements, they only need to access subsets of model elements, and not all specific attributes of those individual elements.

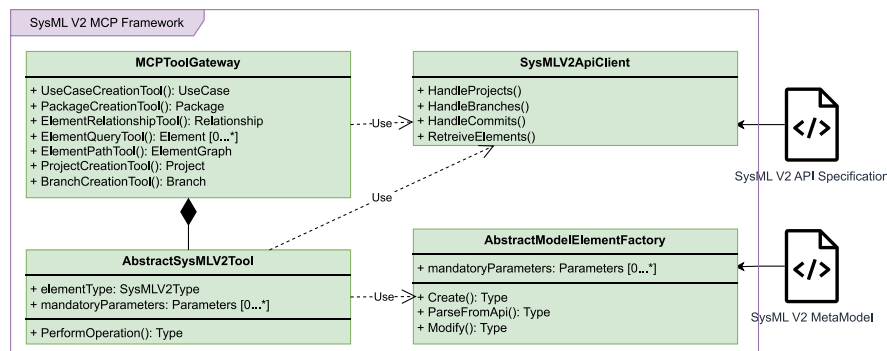


Figure 3. The methodological framework for developing SysML V2 tools in MCP

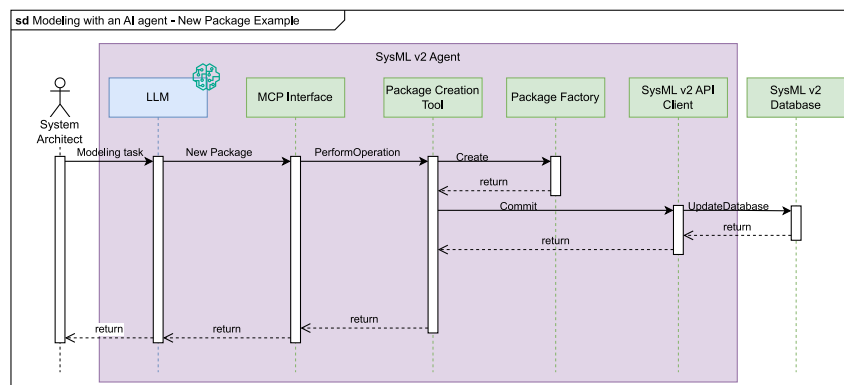


Figure 4. An example of creating packages in SysML v2 with an MCP server

Figure 4 shows an example of a model creation task, and how an MCP tool to handle that task can be implemented using the presented framework. In this example, the LLM calls the MCP interface with the requirement to create a new SysML v2 package. The package creation tool is then activated, which in turn creates a package object, with all the required attributes to it such that there are no syntactic errors when adding this created model to the model database. The tool then commits the package to the SysML v2 database and reports the result back to the LLM through the MCP interface.

The framework presented in this work has been implemented using C#.Net. To demonstrate the functionality discussed here, we used the official SysML V2 API implementation and database in the form of a docker container that interacts with our MCP framework.

To ensure reproducibility of this work, it is made available as an open-source repository¹.

¹ <https://github.com/hni-ase/SysMLV2-mcp>

5. Case studies

To demonstrate our approach to agentic MBSE modeling, we present two case studies based on work that has been performed earlier in (Schleifer et al., 2025) and (Bazzal et al., 2025). In each of these case studies we show how a workflow-based approach to using generative AI in MBSE purely using prompting can be transformed into an agentic approach using the framework presented in section 4. Specifically, these case studies aim to a) show the usage of the framework developed in the previous section, and b) demonstrate the feasibility and benefit of enabling agentic task handling, when compared to workflows that have been used until now.

Note that in figures 5, 6, and 7, we use a UML sequence diagram here to describe the interaction between different components of our framework, not to represent the outcome of the modeling task.

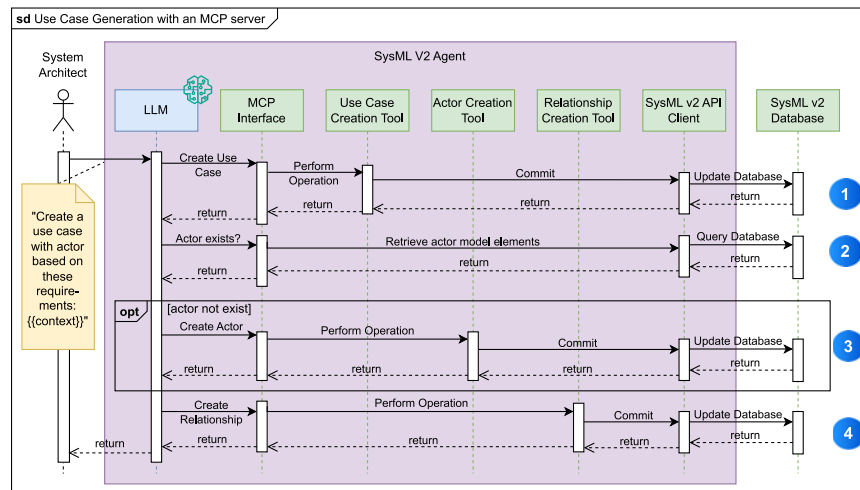


Figure 5. The working principle of SysML v2 use case generator agent presented in case study 1

6. Use case generation

The first case study outlines necessary first steps to transfer the AI-based use case generation method proposed by (Schleifer et al., 2025) into an agentic approach using the MCP server developed in section 4. The originally presented implementation uses several prompts to define fully described use cases and identify related actors based on input requirements to eventually generate the use case diagram. Each prompt requires specific input parameters that are statically provided by the system model. The LLM-generated results are in a JSON format and get parsed into SysML model elements by a custom plugin. Moving towards an agentic reimplementation, we start by providing simpler tools with focus on generating the necessary SysML v2 model elements, despite implementing complex pipelines as MCP server tools.

A detailed description of the implementation is given in Figure 6. For the sake of simplicity, the required model element factories are not shown in the figure. The system architect prompts the LLM to create a new use case with an associated actor based on a set of provided input requirements, e.g., functional requirements specifying the electrical adjustment of the driver's seat. In the agentic framework, the LLM can then decide to call one of the tools provided by the MCP server that creates a new use case. Its inputs are for example the use case name (e.g., "adjust driver seat") and a user-friendly documentation of the use case. After creating the new model element, the SysML v2 API client updates the system model into the database with the element and the successful return message is propagated back to the LLM (1). Next, the LLM decides to invoke a query tool to determine if the required actor "driver" already exists within the system model (2). If the actor model element is not part of the system model yet, the LLM decides to trigger a similar workflow as before to automatically create the missing actor (3). Otherwise, the LLM will directly decide to use the provided MCP relationship creation tool to define an association between the newly created use case "adjust driver seat" and the existing actor "driver" (4). This tool requires the UUID of both the source and target of the relationship as well as its type, here "association." Again, the tool updates the SysML v2 database. With that, the task prompted by the system architect is completed by the agentic framework.

7. Signal optimization

In the second case study, we build upon a signal optimization methodology in vehicle system architectures previously proposed in (Bazzal et al., 2025), which employs a sequential, multi-stage LLM workflow to optimize such architectures by identifying and resolving signal redundancies, overlaps, and semantic ambiguities. In this approach, signals are optimized through retrieving the context of a given set of SysML signals from a vector database, followed by a series of sequential LLM calls, each responsible for a distinct decision, such as detecting duplicates, merging similar signals, or splitting heterogeneous ones. These LLM steps are then followed by a model refactoring suggestion.

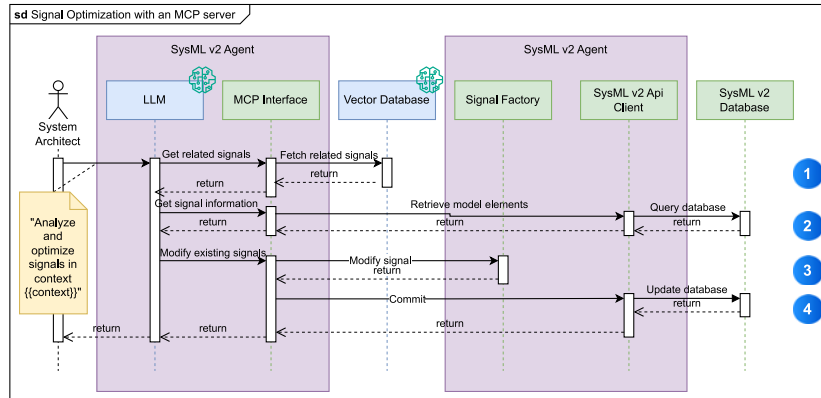


Figure 6. The working principle of a SysML v2 signal optimization agent presented in case study 2

For an agentic reimplementation, the MCP server enables an agentic execution model in which the LLM operates as a decision-making agent, dynamically requesting only the necessary signal context from the vector database when required. Once a decision is made, the MCP tools facilitate direct, programmatic model modification through model retrieval and update tools, eliminating the need for manual prompt orchestration or ad hoc API calls. The LLM can then choose a tool to call from those provided by the MCP server. Here the MCP server provides tools to query SysML v2 model elements, to modify those elements, and a tool that fetches the related context of a given signal from a vector database embedding all signals in the system architecture being optimized.

Figure 7 illustrates the agentic implementation of this optimization task. A prompt describing the task to the LLM is provided by the system architect (1). The LLM then initially chooses to call the vector database to get all signals that are related to the prompt (1). These signals are then used by the LLM to fetch all their related elements from the SysML v2 database (2). Next, the LLM performs the required content evaluation for the context returned from the model retrieval tool and makes an optimization decision regarding the signals (remove duplicate, merge signals, split signals). It then calls the model modification tools to modify the signals as model elements, thereby implementing the optimization decision taken in (2). The MCP server handles the signal modification request by executing the “modify” operation of the signal factory (3), which, if syntactically invalid, would return an error to the LLM such that it could try a different signal modification, performing as an additional layer of error handling. Finally, the signal modification tool commits the change to the SysML v2 database by calling its API and returns the status to the LLM (4). Depending on the prompt, the LLM can summarize the changes being made in natural language and return that summary to the system architect. Note that in this scenario, each tool can be called multiple times to perform a chain of operations that depend on each other, such as splitting a signal first, realizing that some of the split signals are duplicates, and consequently removing these duplicates. Additionally, tools provided by the MCP server that can handle branches and commits can be used to create multiple versions of the model that include multiple optimization suggestions, which can later be reviewed by the system architect, thus maintaining the human in the loop aspect of this agentic approach.

8. Discussion

Having demonstrated the ability of our framework to enable the agentic execution of MBSE tasks defined in both case studies, we discuss in this section the feasibility, benefits, limitations, and derived design principles of this work. First, the case studies show that our framework can be used to adapt existing

MBSE tasks that have been otherwise automated to the agentic approach. This addresses a) the sequential nature required to perform each of these tasks, and b) the ability to handle deviations in the MBSE task responding to different contexts where these tasks are performed in a system architecture. This has been done in both cases by identifying the elements that need to be queried, modified or created with each task, building the MCP tools required for each of these elements, adding tools to communicate with other sources out of the SysML V2 model (e.g. the use of a vector database in case study 2), and finally letting the agent decide the overall steps to take while performing each task. While the feasibility of the agentic automation of MBSE tasks using our framework can be shown in terms of software architecture, we identify the following advantages of this implementation. First, it enables adaptive, context-sensitive reasoning: instead of predefining a fixed sequence of actions, the agent can dynamically explore the model state, query relevant information on-demand, and adjust its strategy in response to complex or ambiguous element relationships. For example, the LLM in case study 2 can adjust the signal optimization decisions being taken for a given signal dynamically based on previous optimization decision of the same signal and changing context. Second, the agentic approach to MBSE task automation reduces the burden of prompt engineering on the system architect's part, since LLMs no longer need to encode infrastructure or tool-level knowledge such as how to retrieve or update a model element via an API. Instead, it can focus on higher level architectural reasoning such as which signal optimization decision to take under which circumstances in case study 2, or which actor is more appropriate for a given use case in case study 1. Third, the agentic approach supports iterative refinement: if a proposed change leads to unexpected consequences (e.g., breaking a dependency), the agent can re-evaluate, re-query, and adjust its strategy – which is inherently difficult in a linear, one-way prompt chain. Given these benefits, the framework is demonstrated through the case study to be fulfilling the two requirements identified in [section 1](#), namely the ability to automate MBSE tasks both in an agentic, and interoperable manner. However, the framework presented here has limitations. First, despite the popularity of MCP as an interoperability enabler for agentic AI, it is not the only technology that enables it. Second, this work assumes that a modelling task can be automated by a single agent, which is not always the case for more complex or higher-level tasks. Furthermore, the framework presented in [section 4](#) is tailored towards building MCP tools for SysML v2. Using other modelling languages where a tool-agnostic unified API does not exist, would require changes to our framework, especially in terms of choosing the granularity of the MCP tools provided to the LLM, and their implications in terms of the split of modeling aspects handled deterministically by the MCP server, and architectural aspects handled by the LLM. Finally, the limitation on the number of MCP tools a server can provide, without degrading the reasoning capability of an LLM that can use it, limits the number of SysML model elements that can be handled by a given MCP server implemented using our framework. Given these benefits, and limitations, and following ([Möller et al., 2020](#)) we reflect on the framework and development process leading to it, and extract the **design principles guiding the development of agentic MBSE solutions** as follows:

First, the choice of which tools to provide to the LLM must be strategic. SysML V2 as a language has more than 450 metamodel elements. Creating tools for the creation, modification, and fetching of each one of these elements would explode the number of MCP tools an LLM has to choose from. Therefore, identifying the metamodel elements required for automating a given MBSE task is essential in both minimizing the development complexity of the MCP server, while not losing the LLM's ability to perform actions on the model dynamically, i.e. its agentic capability. Two design principles follow from this. A) The implemented tools should be tailored to the task being done by an agent, B) for complex tasks requiring a large variety of metamodel elements, the tools should be split into multiple MCP servers based on the architectural aspects handled by those metamodel elements (e.g. tools for signals, interfaces, port definitions, and usages should belong to the same MCP server), and C) metamodel elements that share a common ancestor (e.g. Part usages, and port usages are both of type usage) should be combined in one generic tool that accepts the ancestor as input and takes the correct action based on the concrete element being passed to the MCP tool. Additionally, a final design principle is that error handling should maintain the separation of concerns between the LLM and the tool. An error that occurs based on erroneous LLM input to a tool should be verbosely described, therefore helping the LLM to correct the erroneous input in the next iteration.

9. Conclusion and outlook

This paper presents a framework that enables agentic MBSE with the use of MCP and SysML v2 as a modeling language. The framework serves as a foundation to further develop tools to be used by LLMs in an interoperable fashion for specific MBSE tasks such as model creation, modification, and analysis. To demonstrate the framework, we applied it to two use cases that have been converted for an agentic approach. This demonstration shows the capability of this framework to adapt and integrate existing generative AI-enabled MBSE applications, in a tool agnostic way by using a new modeling language such as SysML V2. It also shows that systems engineering tasks can be automated dynamically, instead of using statically defined workflows with dedicated LLM steps. The implementation leading to this paper has been performed to demonstrate the suitability of MCP concepts to perform modeling and systems engineering tasks. Eventually, product development of complex systems benefits from a reduced time to market by this work, through enabling faster iterations of the system architecture, and enhanced consistency of the resulting system models.

References

- Bader, E., Vereno, D., & Neureiter, C. (2024). Facilitating User-Centric Model-Based Systems Engineering Using Generative AI. *Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering - Volume 1: MBSE-AI Integration*, 371–377. doi:[10.5220/0012623200003645](https://doi.org/10.5220/0012623200003645)
- Bajaj, M., Friedenthal, S., & Seidewitz, E. (2022). Systems Modeling Language (SysML v2) Support for Digital Engineering. *INSIGHT*, 25(1), 19–24. <https://doi.org/10.1002/inst.12367>
- Bazzal, M., Lungu, A., Kruse, B., Koldewey, C., & Dumitrescu, R. (2025). Towards an AI-Enabled Signal Optimization Methodology in Vehicle Architecture Models. *2025 IEEE International Symposium on Systems Engineering (ISSE)*, 1–8. doi:[10.1109/ISSE65546.2025.11369992](https://doi.org/10.1109/ISSE65546.2025.11369992)
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). *Language Models are Few-Shot Learners* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2005.14165>
- Delligatti, L. (2014). *SysML distilled: A brief guide to the systems modeling language*. Addison-Wesley.
- Ehtesham, A., Singh, A., Gupta, G. K., & Kumar, S. (2025). A survey of agent interoperability protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). <https://arxiv.org/abs/2505.02279v2>
- Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions (No. arXiv:2503.23278). arXiv. <https://doi.org/10.48550/arXiv.2503.23278>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models*. <https://web.stanford.edu/~jurafsky/slp3/>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Version 4). arXiv. <https://doi.org/10.48550/ARXIV.2005.11401>
- Möller, F., Guggenberger, T. M., & Otto, B. (2020). Towards a Method for Design Principle Development in Information Systems. In *Lecture Notes in Computer Science* (pp. 208–220). Springer International Publishing. https://doi.org/10.1007/978-3-030-64823-7_20
- Mollahassani, D., Becker, M., Eickhoff, T., Pickel, J., Goetz, S., Wartzack, S., & Göbel, J. C. (2025). Framework for circular AI-driven model-based systems engineering. *Proceedings of the Design Society*, 5, 1883–1892. doi:[10.1017/pds.2025.10202](https://doi.org/10.1017/pds.2025.10202)
- Peffer, K., Tuunanen, T., Rothenberger, M., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *J. Manage. Inf. Syst.*, 24(3), 45–77. doi:[10.2753/MIS0742-1222240302](https://doi.org/10.2753/MIS0742-1222240302)
- Poulsen, V. V., Guertler, M., Eisenbart, B., & Sick, N. (2025). Advancing systems engineering with artificial intelligence: a review on the future potential, challenges and pathways. *Proceedings of the Design Society*, 5, 359–368. doi:[10.1017/pds.2025.10050](https://doi.org/10.1017/pds.2025.10050)
- Schleifer, S., Lungu, A., Kruse, B., Goetz, S., & Wartzack, S. (2025). Large Language Model-based Generation of Use Case Diagrams from Requirement Specifications. *INCOSE International Symposium*, 35(1), 261–275. <https://doi.org/10.1002/iis2.70015>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need* (No. arXiv:1706.03762). arXiv. <https://doi.org/10.48550/arXiv.1706.03762>
- Walden, D. D., Roedler, G. J., Forsberg, K., Hamelin, R. D., Shortell, T. M., & International Council on Systems Engineering (Eds.). (2015). *Systems engineering handbook: A guide for system life cycle processes and activities* (4th edition). Wiley.